



Универзитет „Св. Кирил и Методиј“ во Скопје
**ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И
КОМПЈУТЕРСКО ИНЖЕНЕРСТВО**

Проектна задача

Повеќеслојно автоматизирано тестирање на ParaBank (Postman API, Selenium и Playwright UI)

Предмет: Софтверски квалитет и
тестирање

Изработила: Ева Ѓорѓевиќ, 231169

Датум: Јули 2026

Репозиториум:

<https://github.com/evikj/ParaBank-Testing>

Содржина:

1. Вовед	3
2. Алатки и технологии (Tools & Technologies)	3
3. Имплементирани тест сценарија.....	4
4. Управување со податоци (Data Management).....	6
4.1. Техники на пренос на податоци (Data Chaining)	6
4.2. Пример за примена кај интеграциските сценарија.....	6
4.3. Придобивки за квалитетот	6
5. Playwright (Нова алатка)	6
5.1. Playwright Trace Viewer	6
5.2. Клучни функционалности искористени во проектот:	7
5.3. Резултати:.....	7
Репрезентативни функционални сценарија (За споредба):.....	7
E2E 01: Full lifecycle of a new savings account.....	7
E2E-02: Loan processing and immediate fund transfer	9
6. Резултати и пронајдени дефекти	11
6.1. Статистика на извршување	11
6.2. Извештај за пронајдени дефекти (Defect Reports).....	12
BUG-01: Unbounded Overdraft Allowed via Transfer API	12
BUG-02: Stock Service Integration Failure (Company Name Resolution)	13
BUG-03: Insecure Session Termination (Manual Discovery)	13
6.3. Cross-Browser Резултати (Playwright)	14
7. Заклучок и споредба на алатките	14
7.1. Споредба: Selenium vs. Playwright.....	14
7.2. Согледувања при работењето	15
7.3. Заклучок.....	15

1. Вовед

Целта на овој проект е примена на напредни техники и алатки за обезбедување на софтверски квалитет преку практично тестирање на апликацијата **ParaBank**. Проектот опфаќа повеќеслојно тестирање, вклучувајќи го API слојот и корисничкиот интерфејс (UI).

Како предмет на тестирање беше избрана ParaBank демо апликацијата поради нејзината комплексност во делот на финансиските трансакции и асинхроното вчитување на податоци, што овозможи симулација на реални QA предизвици. Во текот на изработката, фокусот беше ставен на:

- **Автоматизација на REST API услугите преку Postman**, со цел верификација на функционалноста на серверот, интегритетот на податоците и однесувањето на системот при позитивни и негативни тест-сценарија.
- **UI тестирање со Selenium**, со користење на **Page Object Model (POM)** архитектурата за постигнување на чист, модуларен код кој овозможува лесно одржување и стабилно извршување на функционалните тестови.
- **Имплементирање на Playwright** како модерна алатка која не беше дел од наставната програма, со цел да се демонстрираат предностите на автоматското чекање (auto-waiting), Cross-browser тестирањето и напредната дијагностика преку Trace Viewer кај сложени **End-to-End (E2E)** текови.

Проектот се стреми да ја прикаже функционалноста на клучните модули, преку комбинацијата од различни нивоа на тестирање и технологии.

2. Алатки и технологии (Tools & Technologies)

- **Postman:** Користена како примарна платформа за тестирање на API слојот. Преку неа беше извршена автоматизација на тест-сценаријата со помош на JavaScript тест-скрипти, управување со динамички променливи преку Environments и поврзување на барања за комплексна верификација на backend логиката.
- **Java & Selenium WebDriver:** Применети за UI автоматизација на клучните функционални модули. Проектот е структуриран како **Maven** проект, користејќи го **JUnit 5** како *test runner* и **WebDriverManager** за автоматско управување со драјверите за прелистувачи.
- **Playwright (Node.js):** Имплементирана како технологија за **End-to-End (E2E)** тестирање. Оваа алатка овозможи Cross-browser верификација (Chromium и Firefox), а нејзините вградени механизми за **автоматско снимање на видео записи и интерактивни траги (Trace Viewer)** беа клучни за напредна дијагностика и дебагирање на сложените кориснички процеси.
- **GitHub:** Користен за верзионирање на изворниот код и документацијата.

3. Имплементирани тест сценарија

ID	Категорија	Тест Сценарио	Тип	Опис и Цел	Исход / Забелешка
API-01	API	Successful Login	Позитивен	Верификација на REST ендпоинтот за најава на постоечки корисник.	Pass
API-02	API	Invalid Login	Негативен	Обид за најава со погрешна лозинка. Очекуван 4xx статус.	Pass (400)
API-03	API	Create Account	Позитивен	Динамичко креирање на нова сметка преку POST барање.	Pass
API-04	API	Deposit Funds	Позитивен	Депонирање средства на новокреираната сметка.	Pass
API-05	API	Get Invalid Account	Негативен	Барање податоци за сметка со непостоечки ID.	Pass
API-06	API	Overdraft Transfer	Логички	Обид за трансфер на сума поголема од билансот.	FAIL (BUG-01)
API-07	API	Invalid Loan Request	Негативен	Барање кредит со негативни износи или невалидни параметри.	Pass
API-08	API	Account List Integrity	Интеграциски	Проверка дали новата сметка е видлива во листата на клиентот.	Pass
API-09	API	Buy Stock Position	Позитивен	Купување на акции преку инвестицискиот модул.	BLOCKED (BUG-02)
API-10	API	Get Invalid Position	Негативен	Барање на берзанска позиција која не постои.	Pass
API-11	API	Transaction History	Позитивен	Верификација дека трансакцијата е запишана во историјата.	Pass
UI-01	UI (Selenium)	Successful Login	Позитивен	Верификација на најава преку корисничкиот интерфејс.	Pass
UI-02	UI (Selenium)	Unsuccessful Login	Негативен	Проверка на порака за грешка при погрешни кренденцијали.	Pass
UI-10	UI (Selenium)	Logout & Session	Безбедносен	Проверка дали сесијата се уништува по одјава (URL заштита).	FAIL(BUG-03)

UI-03	UI (Selenium)	Open New Account	Позитивен	Автоматизирано отворање на сметка преку POM архитектура.	Pass
UI-08	UI (Selenium, Playwright)	Balance Consistency	Податоци	Проверка дали билансот е ист во Overview и во Details.	Pass
UI-07	UI (Selenium)	Create & Find Trans.	Интеграциски	Креирање трансакција и нејзино наоѓање преку филтри.	Pass
UI-04	UI (Selenium, Playwright)	Loan Approved	Логички	Барање кредит со 20% учество преку UI.	Pass
UI-05	UI (Selenium)	Loan Denied	Логички	Барање кредит со недоволно учество преку UI.	Pass
UI-06	UI (Selenium, Playwright)	Bill Pay Process	Позитивен	Целосно пополнување на комплексна форма за плаќање.	Pass
UI-11	UI (Selenium)	Empty Form Validation	Usability	Проверка на клиентска валидација за празни полиња.	Pass
UI-09	UI (Selenium)	Update Contact Info	Usability	Ажурирање на полиња за корисничките информации	Pass
E2E-01	Playwright	New Customer Loop	E2E	Login -> Create Acc -> Bill Pay -> Verify History.	Pass
E2E-02	Playwright	Loan & Redistribution	E2E	Login -> Get Loan -> Transfer Funds -> Logout.	Pass

- **REST API (Postman):** Фокусот беше ставен на верификација на backend логиката и валидација на HTTP статусите. Преку овој слој беа детектирани најкритичните дефекти поврзани со **финансискиот интегритет** (неограничен минус) и дефектите во **инвестицискиот модул** (Stock Service).
- **UI Selenium (POM):** Со примена на *Page Object Model* архитектурата беше овозможена стабилна верификација на функционалноста на формите и навигацијата. Покрај функционалните проверки, овој слој беше клучен за идентификување на **безбедносен пропуст** при терминирање на корисничките сесии (Logout vulnerability).
- **UI Playwright (E2E):** Како нова алатка, Playwright беше искористен за сложени текови (**Workflows**) каде што повеќе функционалности се тестираат како една целина. Тука е демонстрирана моќта на **Auto-waiting** и **Trace Viewer**.

4. Управување со податоци (Data Management)

Поради природата на ParaBank апликацијата, каде што секое отварање на нова сметка генерира различен и непредвидлив ID број, беше неопходно да се имплементира основно ниво на динамичко управување со податоците. Наместо користење на надворешни бази (DDT), се фокусиравме на конзистентност на податоците во рамките на самиот тест-циклус.

4.1. Техники на пренос на податоци (Data Chaining)

- **API слој (Postman Environment):** При креирање на нова сметка, ID-то од JSON одговорот автоматски се зачувува како променлива во *Environment* сегментот. Оваа вредност потоа автоматски се вбригува во URL-патеките на следните барања за депозит и трансфер.
- **UI слој (Dynamic Extraction):** Во Selenium и Playwright сценаријата, тест-скриптите се конфигурирани да го „прочитаат“ новогенерираниот број на сметка директно од корисничкиот интерфејс (на пр. од потврдата за отворена сметка) и да го зачуваат во локална променлива за понатамошна употреба во паѓачките менија.

4.2. Пример за примена кај интеграциските сценарија

Кај сценаријата **UI-07** (Selenium) и **E2E-01** (Playwright), за да се обезбеди точност на резултатите, беше користен следниов пристап:

1. Дефинирање на специфичен износ (пр. 12.34) на почетокот на тестот.
2. Користење на тој износ при извршување на трансакцијата.
3. Повторно користење на истиот износ како критериум за пребарување во филтрите.

4.3. Придобивки за квалитетот

- **Повторливост:** Тестовите можат да се извршуваат повеќе пати без потреба од мануелно ресетирање на вредностите во кодот.
- **Стабилност:** Се елиминираат паѓањата на тестовите кои би се случиле доколку апликацијата генерира податок кој не е однапред познат за скриптата.

5. Playwright (Нова алатка)

Како дополнителна технологија беше избран **Playwright (Node.js)**. За разлика од традиционалните алатки, Playwright е современа библиотека наменета за брза и робусна автоматизација на прелистувачи, која во овој проект беше искористена за имплементација на **End-to-End (E2E)** сценарија како и споредба на некои UI тестови кои ги направивме во Selenium.

5.1. Playwright Trace Viewer

Trace Viewer е GUI алатка за пост-мортем анализа на извршените тестови. За разлика од обичните логови, Trace Viewer овозможува „патување“ низ секој чекор од тестот.

5.2. Клучни функционалности искористени во проектот:

- **Action Snapshots:** Овозможува преглед на прецизната состојба на DOM дрвото и визуелниот приказ на страницата точно пред, за време и по секоја извршена акција (клик, внес на текст). Ова беше клучно за верификација на преминот од пополнета форма до екранот за потврда кај трансферите.
- **Network Inspection:** Обезбедува целосен увид во сите мрежни повици (XHR/Fetch) кои апликацијата ги прави во позадина. Ова ни овозможи да потврдиме дека UI акцијата за трансфер на средства успешно испратила POST барање до серверот и да го анализираме JSON одговорот без напуштање на тест-извештајот.
- **Source Code Linking:** Trace Viewer директно ја поврзува секоја акција со соодветната линија код во нашите .spec.js фајлови, со што се олеснува процесот на дебагирање на евентуалните падови на тестовите.
- **Execution Video & Screenshots:** Како дополнителен доказ за квалитетот, беа конфигурирани автоматски видео записи од целиот тек на извршувањето, кои се прикачени во документацијата на репозиториумот.

5.3. Резултати:

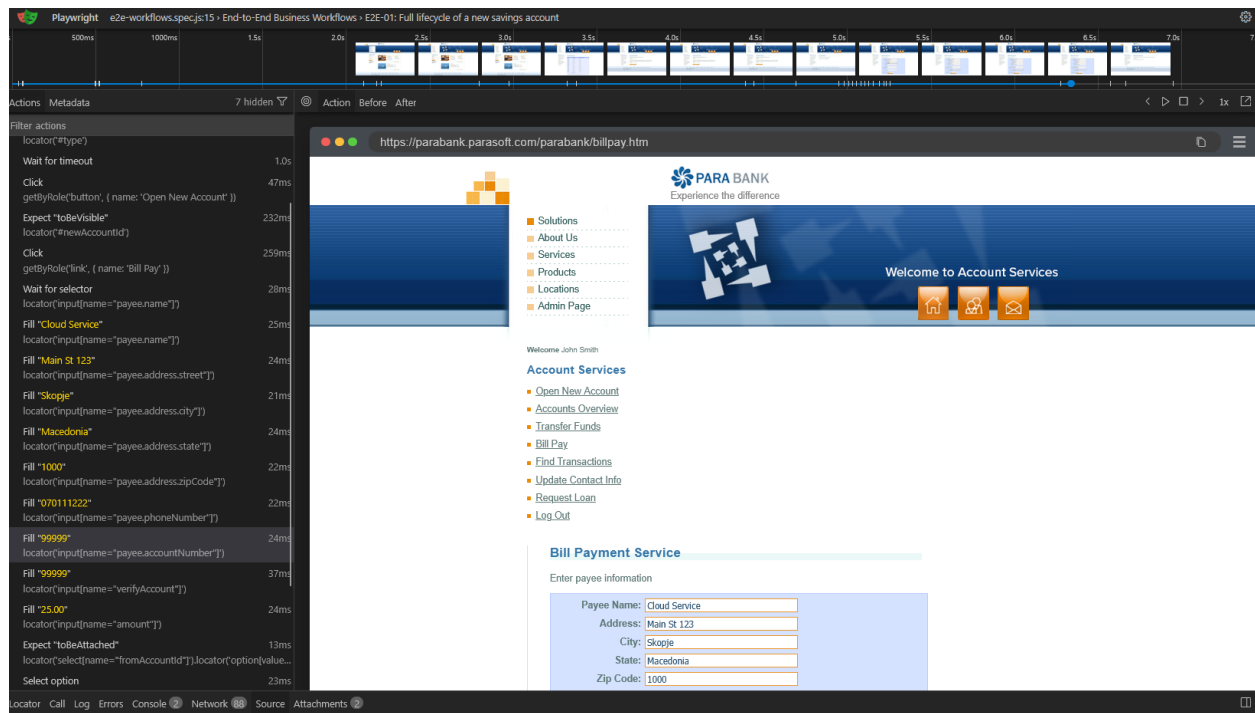
Репрезентативни функционални сценарија (За споредба):

Избравме три специфични сценарија кои претходно беа имплементирани во Selenium (**UI-04**, **UI-06** и **UI-08**). Овие сценарија беа избрани поради нивната техничка тежина (AJAX синхронизација, комплексни форми и мрежни повици), со цел директно да се споредат перформансите и стабилноста на двете алатки.

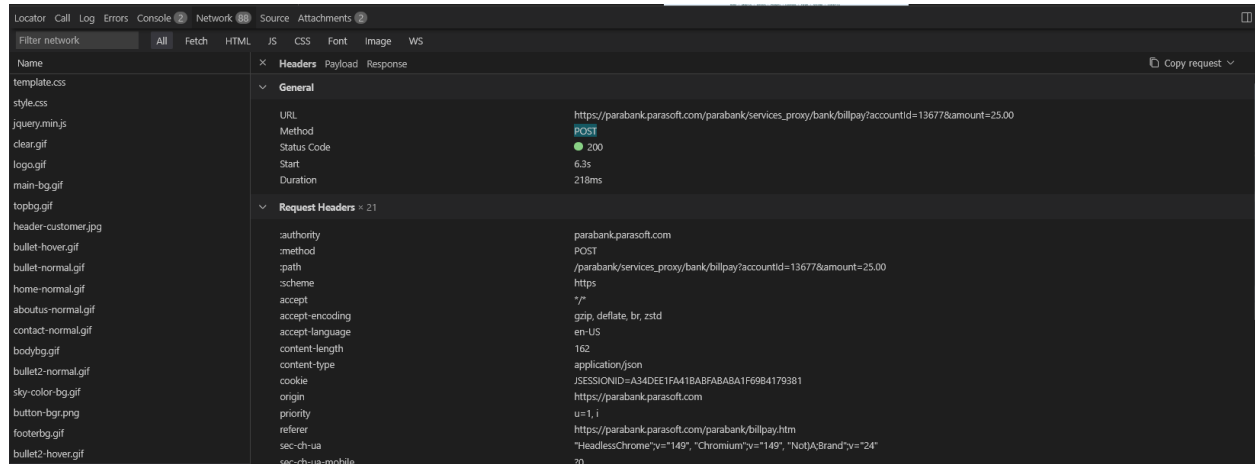
Во репозиториумот, во папката docs/playwright-artifacts, е прикачен фајлот chromium-execution-trace.zip. Овој фајл може да се отвори преку playwright.dev/trace и ги содржи сите метаподатоци од последното успешно извршување на E2E тестовите.

E2E 01: Full lifecycle of a new savings account

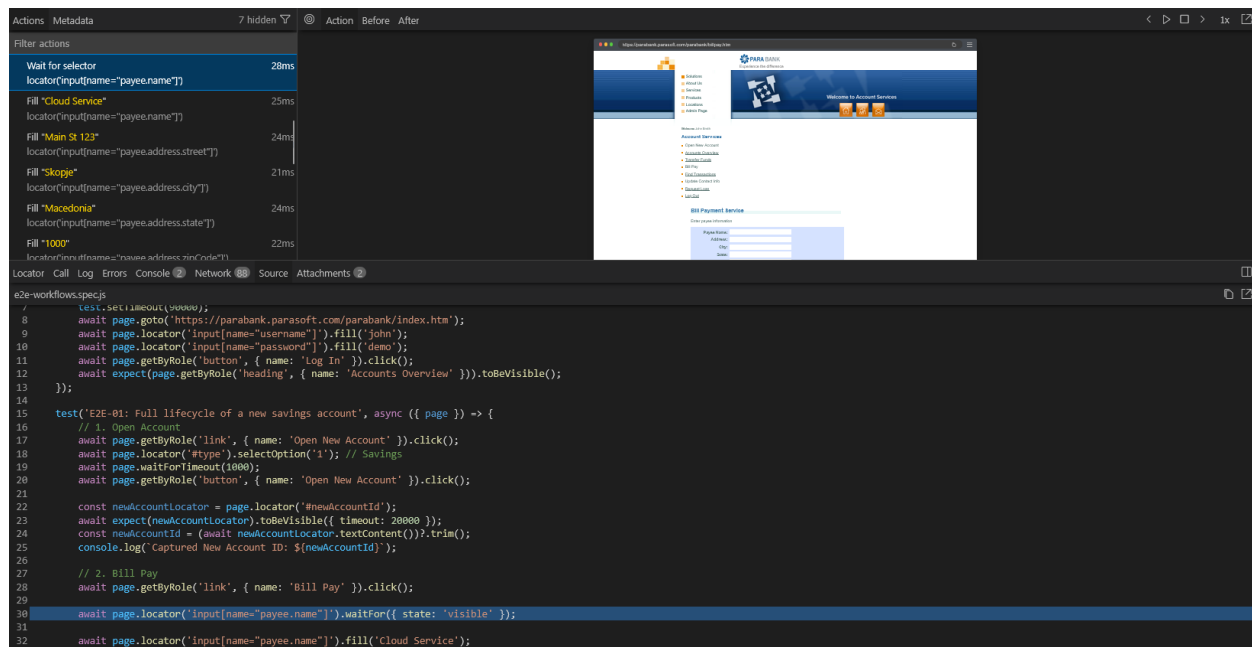
- **Логички тек:** Login → Open New Account (Savings) → Bill Pay → Find Transactions
- **Опис:** Овој тест ја верификува вертикалната интеграција на три различни модули. Започнува со креирање на нова сметка, потоа системот динамички го користи тој нов број на сметка за да изврши плаќање кон надворешен примач (Bill Pay). На крајот, тестот се навигира до пребарувачот на трансакции за да потврди дека плаќањето е успешно прокнижено во историјата на новата сметка.
- **Квалитетен аспект:** Верификација на **Data Flow** (проток на податоци) помеѓу модулите кои работат со иста база во реално време.



Сл. 1 Timeline and actions E2E-01



Сл. 2 Network Tab E2E-01



Сл. 3 Source Tab E2E-01

На Сл. 1 е прикажан **Trace Viewer** интерфејсот каде јасно се гледаат сите чекори на E2E сценариото.

- **Визуелна ретроспектива:** Преку горната временска линија, можеме да ја видиме состојбата на интерфејсот во секој момент од извршувањето.
- **Интеграциска верификација:** Во Network табот (Сл. 2) е документиран POST повикот кон серверот за трансфер на средства, што потврдува дека UI акцијата успешно ја активирала backend логиката.
- **Traceability:** Source табот (Сл. 3) овозможува директно поврзување на секоја линија од тест-кодот со конкретна визуелна промена на екранот.

E2E-02: Loan processing and immediate fund transfer

- **Логички тек:** Login → Request Loan (Approved) → Transfer Funds → Logout
- **Опис:** Фокусот на ова сценарио е на финансиската логика на апликацијата. Тестот аплицира за кредит со соодветно учество за да добие статус „Approved“. Веднаш по одобрувањето, се врши трансфер на дел од одобрените средства од кредитната сметка на примарната сметка на корисникот. Сценариото завршува со безбедносна одјава.
- **Квалитетен аспект:** Тестирање на **Business Rules** (бизнис правила за одобрување) и способноста на системот да управува со трансакции помеѓу сметки со различен статус.

The screenshot shows the Cypress test runner interface. On the left, a list of actions is displayed with their durations:

- locator("#amount")
- Fill "100" (25ms)
- locator("#downPayment")
- Click (60ms)
- getByRole("button", { name: 'Apply Now' })
- Expect "toHaveText" (350ms)
- locator("#loanStatus")
- Expect "toBeVisible" (8ms)
- getByRole("heading", { name: 'Loan Request Processed' })
- Click (281ms)
- getByRole("link", { name: 'Transfer Funds' })
- Expect "toBeAttached" (238ms)
- locator("#fromAccountId").locator("option[value='13788']")
- Select option (16ms)
- locator("#fromAccountId")
- Fill "100" (19ms)
- locator("#amount")
- Click (56ms)
- getByRole("button", { name: 'Transfer' })
- Expect "toBeVisible" (330ms)
- getByRole("heading", { name: 'Transfer Complete' })
- > After Hooks (281ms)

On the right, a browser window shows the Para Bank application. The URL is `https://parabank.parasoft.com/parabank/requestloan.htm`. The page displays a sidebar with navigation links (Solutions, About Us, Services, Products, Locations, Admin Page) and a main content area with a "Welcome to Account Services" message and an "Apply for a Loan" form.

Below the browser window, the Cypress command log is visible, showing the following code:

```

e2e-workflows.spec.js
>>
56 test('E2E-02: Loan processing and immediate fund transfer', async ({ page }) => {
57   // 1. Get Loan
58   await page.getByRole('link', { name: 'Request Loan' }).click();
59   await page.locator("#amount").fill('500');
60   await page.locator("#downPayment").fill('100');
61   await page.getByRole('button', { name: 'Apply Now' }).click();

```

Сл. 4 Timeline and actions E2E-02

The screenshot shows the Chrome DevTools Network tab. The selected request is a POST request to `https://parabank.parasoft.com/parabank/services_proxy/bank/transfer?fromAccountId=13788&toAccountId=12345&amount=100`. The request status is 200, and the duration is 3.4s.

The request headers are as follows:

- authority: parabank.parasoft.com
- method: POST
- path: /parabank/services_proxy/bank/transfer?fromAccountId=13788&toAccountId=12345&amount=100
- scheme: https
- accept: text/plain,*/*; q=0.01
- accept-encoding: gzip, deflate, br, zstd
- accept-language: en-US
- content-length: 0
- cookie: JSESSIONID=060D2DB048BDB4E4C855C6093FAF9557
- origin: https://parabank.parasoft.com
- priority: u=1,i
- referer: https://parabank.parasoft.com/parabank/transfer.htm
- sec-ch-ua: "HeadlessChrome";v="149", "Chromium";v="149", "Not(A)Brand";v="24"
- sec-ch-ua-mobile: ?0
- sec-ch-ua-platform: "Windows"
- sec-fetch-dest: empty
- sec-fetch-mode: cors
- sec-fetch-site: same-origin
- user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.7827.55 Safari/537.36
- x-requested-with: XMLHttpRequest

The response headers are as follows:

- cf-cache-status: DYNAMIC

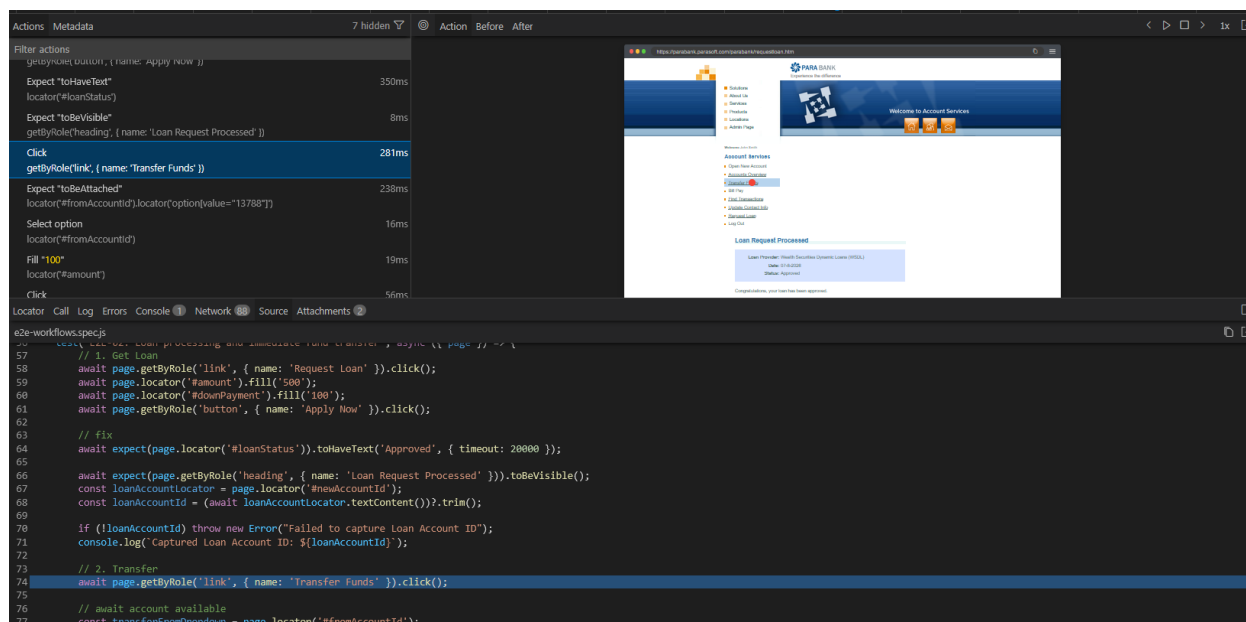
The screenshot shows the response payload for the selected request. The response is a JSON object with the following structure:

```

{
  "responseDate": 1783519814856,
  "loanProviderName": "Wealth Securities Dynamic Loans (WSDL)",
  "approved": true,
  "accountId": 13788
}

```

Сл. 5 Network Tab



Сл. 6 Source Tab

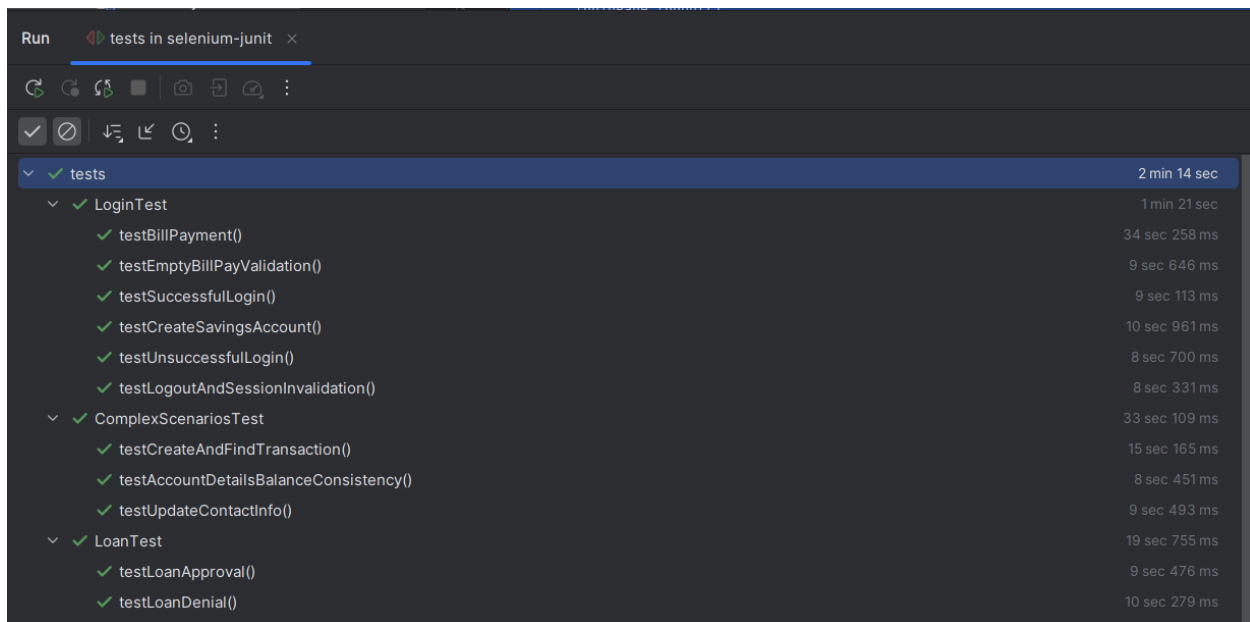
За дополнителна верификација, беше анализирано сценариото за одобрување кредит преку Trace Viewer (Слики [4,5,6]).

- **Прецизност на мрежата:** Во мрежниот преглед е документиран моментот на синхронизација каде UI чека на логичкиот одговор од Loan Processor сервисот.
- **Инспекција на објекти:** Преку Source табот е демонстриран начинот на кој се врши динамичко зачувување на новогенерираниот број на сметка од кредитот, што овозможува понатамошен трансфер на средства во истиот тест-циклус.

6. Резултати и пронајдени дефекти

6.1. Статистика на извршување

Категорија	Вкупно сценарија	Успешни (Pass)	Неуспешни (Fail)	Блокирани (Blocked)
REST API (Postman)	11	9	1	1
UI (Selenium POM)	11	10	1	0
E2E (Playwright)	2	2	0	0
ВКУПНО	24	21	2	1



Сл. 7 Успешно извршување на Selenium тест-пакетот во IntelliJ

6.2. Извештај за пронајдени дефекти (Defect Reports)

Како резултат на автоматизираното тестирање, беа идентификувани следниве критични дефекти:

BUG-01: Unbounded Overdraft Allowed via Transfer API

- **Резиме:** Ендпоинтот за трансфер на средства дозволува произволни негативни биланси поради недостаток на верификација на лимитот за дозволен минус.
- **Класификација:** Business Logic Flaw / Input Validation Failure
- **Сериозност:** Критична (Critical)
- **Контекст:** Иако тековните сметки (Checking) се дизајнирани да поддржуваат мали негативни биланси (overdraft), системот не спроведува максимално дозволен праг, овозможувајќи невалидна состојба на податоците.
- **Чекори за репродукција (API):**
 - Автентикација на сесија преку Postman со стандарден кориснички профил.
 - Креирање на POST барање до ендпоинтот за трансфер: POST /parabank/services/bank/transfer.
 - Внесување параметри кои присилуваат износ што драстично го надминува билансот:

```
{
  "fromAccountId": "13677",
  "toAccountId": "12345",
  "amount": 9999999
}
```

- Испраќање на барањето.
- **Очекуван резултат:** Системот треба да врати статус за грешка (на пр. 400 Bad Request или 422 Unprocessable Entity) со порака дека трансакцијата го надминува лимитот.

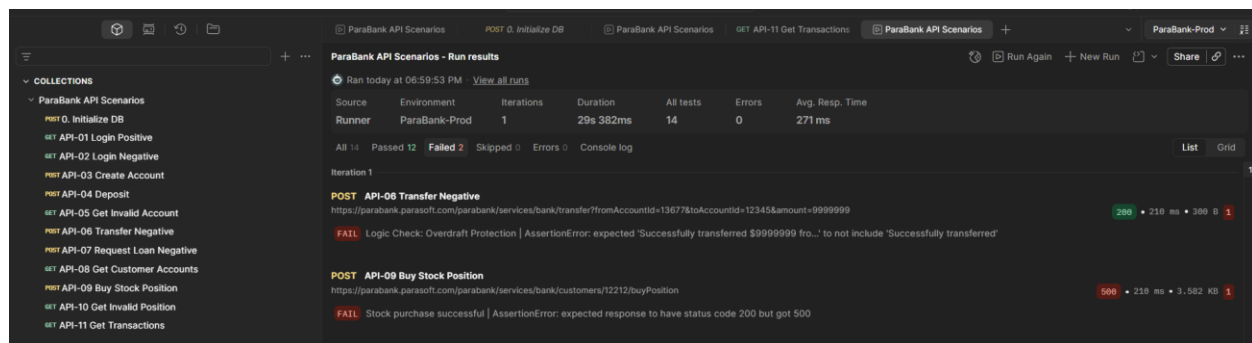
- **Вистински резултат:** 200 OK. Трансферот е успешно процесирен, доведувајќи ја сметката до биланс од -\$9,999,000+.

BUG-02: Stock Service Integration Failure (Company Name Resolution)

- **Резиме:** Инвестицискиот модул не успева да ги процесира купувањата на акции поради неможност за пронаоѓање на имињата на компаниите во базата.
- **Класификација:** Integration Defect
- **Сериозност:** Висока (High) - целосно ја блокира функционалноста за тргување.
- **Контекст:** Системот враќа "null" за имиња на компании дури и за симболи кои се наведени како поддржани во официјалната Swagger документација.
- **Чекори за репродукција (API/Swagger):**
 1. Навигација до инвестицискиот сервис преку Postman или Swagger UI.
 2. Повикување на POST ендпоинтот /buyPosition со валидни влезни податоци:

URL: .../buyPosition?accountId=13566&symbol=PRST&shares=1&pricePerShare=25.50

3. Анализа на JSON одговорот од серверот.
- **Очекуван резултат:** Статус 200 OK со целосни детали за новокреираната позиција (Position ID, Company Name).
 - **Вистински резултат:** Статус **500 Internal Server Error**. Системот враќа порака за грешка: Could not buy position... Company Name: null. Ова укажува на дефект во интеграцијата помеѓу API слојот и Stock-сервисот.



Сл. 8 Визуелна потврда за API тестови

BUG-03: Insecure Session Termination (Manual Discovery)

- **Резиме:** Апликацијата не врши соодветна инвалидација на сесијата на серверска страна, што овозможува неовластен пристап до приватни податоци по одјава.
- **Класификација:** Security Vulnerability / Access Control
- **Статус во автоматизација:** FAIL (Успешно репродуциран преку Selenium).
- **Опис на дефектот:** Иако копчето за одјава визуелно го пренасочува корисникот, сесискиот токен останува активен. Со воведување на чекор за навигација на назад

(`driver.navigate().back()`) во Selenium скриптата, беше потврдено дека сензитивните податоци на Overview страната се сè уште достапни.

- **Објаснување на резултатот:** Иако Selenium скриптата потврди дека системот правилно пренасочува кон Login страната при свежо вчитување на URL-то, мануелната верификација откри дека сесиските токени не се инстантно инвалидирани на серверска страна.
- **Класификација:** Security Vulnerability
- **Чекори за репродукција (Selenium):**
 1. Најава во системот со валидни кредитенцијали (`john/demo`).
 2. Кликнување на линкот "Log Out".
 3. Извршување на наредба за враќање на претходна страница во прелистувачот.
 4. Проверка на содржината за присуство на балансот на сметките.

• **Очекуван резултат:** По одјава, секој обид за враќање на заштитена страница мора да биде блокиран и корисникот да биде пренасочен кон Login формата.

• **Вистински резултат:** Тестот паѓа (**Assertion Error**) бидејќи системот ги прикажува податоците на корисникот и по одјавата, докажувајќи дека сесискиот токен (`JSESSIONID`) е сè уште активен.

6.3. Cross-Browser Резултати (Playwright)

Беше спроведено тестирање на истите E2E сценарија на различни платформи за да се осигура конзистентност на корисничкото искуство:

- **Chromium (Chrome/Edge):** 100% Успешност.
- **Firefox:** 100% Успешност.
- **Webkit (Safari):** Блокирано

По првичните извршувања, донесовме одлука за исклучување на WebKit engine од финалните тестови. Техничка анализа покажа дека WebKit на Windows покажува значително зголемено доцнење при процесирање на асинхроните барања на ParaBank. Тоа резултира со чести прекини во мрежната комуникација, десинхронизација на сесиските колачиња и појава на *Race Conditions*, што предизвикуваше лажни негативни резултати (False Negatives) поради надминување на стандардните тајмаути на Playwright. Бидејќи ова однесување е директна последица на познатите инфраструктурни ограничувања на WebKit емулацијата под Windows ОС, а не на реален дефект во бизнис логиката на ParaBank, беше изоставено понатамошно извршување.

7. Заклучок и споредба на алатките

7.1. Споредба: Selenium vs. Playwright

При имплементација на истите сценарија во двете алатки, ги воочивме следниве разлики:

Карактеристика	Selenium WebDriver (Java)	Playwright (Node.js)
Синхронизација со AJAX	Бараше обемно користење на WebDriverWait и специфични услови (на пр. чекање додека текстот не е празен) за да се избегнат грешки кај бавните ParaBank елементи.	Овозможи многу постабилно извршување преку Auto-waiting , иако кај специфични случаи (како избор на сметка) беше потребна дополнителна логика со <code>toBeAttached</code> .
Брзина на развој	Побавен развој поради потребата од рачно дефинирање на секоја Page Object класа и комплексна Maven конфигурација.	Многу побрзо благодарение на codegen функцијата за автоматско генерирање на тест-чекори директно од прелистувачот.
Дијагностика	Ограничена на логови во конзолата и рачни скриншоти.	Напреден <i>Trace Viewer</i> и автоматски видео записи за секој тест.
Cross-browser	Потребна е посебна конфигурација за секој драјвер (Chrome, Gecko).	Вградена поддршка за сите прелистувачи со минимални промени во конфигурацијата.

7.2. Согледувања при работата

- **Важност на иницијализацијата:** Тестирањето на динамична апликација како ParaBank бара чиста состојба на податоците пред секој циклус. Употребата на `/initializeDB` беше клучна за повторливост на резултатите.
- **Предизвици при синхронизација на асинхрони системи:** Преку споредбата на Selenium и Playwright, беше евидентно дека најголемата пречка за стабилна автоматизација е латенцијата на серверот и асинхроното вчитување на компонентите (AJAX). Искуството покажа дека изборот на **стратегија за чекање (Waiting Strategy)** е поважен од самата алатка.
- **Критичка анализа:** Еден од најзначајните заклучоци е дека „зелениот“ статус на тестот не е апсолутен индикатор за квалитет. Сценариото **BUG-01** (неограничен минус) покажа дека системот може да биде функционално исправен (да извршува трансфер), но **логички дефектен** (да не ги почитува бизнис правилата).
- **Поврзаност помеѓу API и UI слоевите:** Проектот демонстрираше дека квалитетот мора да се набљудува интегрално. Одредени пропусти во безбедноста на сесијата и интегритетот на податоците беа многу поефикасно детектирани на API слојот, додека нивното влијание врз корисничкото искуство беше валидирано преку UI слојот.

7.3. Заклучок

Проектот успешно ги потврди клучните функционалности на ParaBank, но истовремено ги изложи и неговите критични слабости во делот на бизнис логиката и безбедноста. Комбинацијата на трите алатки (Postman, Selenium, Playwright) овозможи преглед на квалитетот на системот, со реализацијата на тест-сценаријата и деталната дијагностика на пронајдените дефекти, проектната задача ги исполни поставените цели. Стекнатите искуства од овој процес нудат вредна основа за понатамошен развој во областа на автоматизираното тестирање и примена на високи стандарди за квалитет во софтверското инженерство.